Task C2 — Preprocessing Pipeline

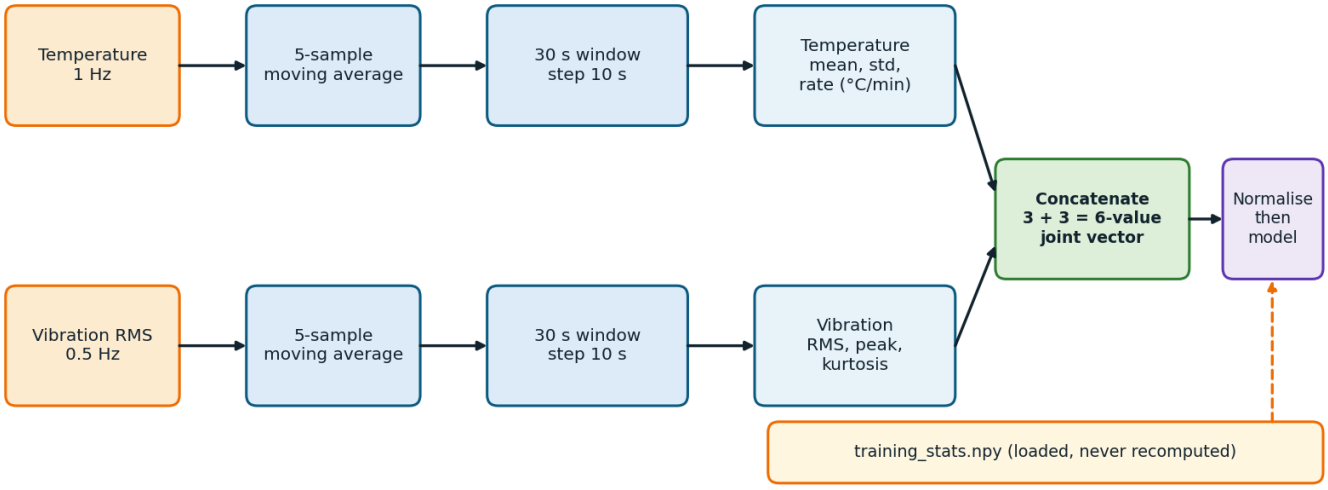
Phase 2 Report section | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component C, Module 3

At a glance

Item	Result
Pipeline	5-sample moving average → 30 s windows, 10 s step → 6 features → z-score with saved statistics
Statistics file	`training_stats.npy`, shape (2, 6), from 10 min of clean Normal-class data (57 windows); loaded at runtime, never recomputed
Mandatory experiment	Correct statistics 99.67%; shifted +3σ 98.38% (−1.29 points); shifted −3σ 91.28% (−8.38 points)
Detection delay	Mean 18.2 s for drift (max 26 s), 6.6 s for vibration; inside the 90 s limit
Verification	32 preprocessing tests pass (plus the 23 simulator tests from Task C1)

1 Pipeline

Each sensor is filtered, windowed and summarised on its own, then fused at feature level



Stage	Specification	Implementation
Filtering	5-sample moving average on temperature and vibration	Causal, one filter per stream. The first four outputs average the samples seen so far, so no sample is lost at start-up.
Windowing	30 s sliding window, step 10 s	Window [s, s+30) is emitted when a sample with timestamp ≥ s+30 arrives: 30 temperature and 15 vibration samples. A window with under 80% of its samples is skipped and counted, never fabricated.
Features	6 values	Three per sensor, extracted independently (Section 2).
Fusion	One joint vector (Task C3)	The two 3-value blocks are concatenated: temperature first, then vibration.
Normalisation	Saved mean and std	z = (x − mean) ÷ std with values loaded from training_stats.npy.

2 The six features and their Normal-class statistics

#	Feature	Definition	Mean	Std
0	temp_mean	Mean of the 30 filtered samples (°C)	4.004	0.055
1	temp_std	Population standard deviation (°C)	0.123	0.026
2	temp_rate_c_per_min	Least-squares slope over the window × 60 (°C/min)	0.002	0.362
3	vib_rms	√(mean of x²) over the filtered samples (g)	0.4454	0.0105
4	vib_peak	max |x| over the filtered samples (g)	0.4717	0.0117
5	vib_kurtosis	Excess kurtosis, m4 ÷ m2² − 3 (a Gaussian gives about 0)	−0.805	0.530

The vibration input is the 0.5 Hz RMS stream from Task C1, so "vibration RMS" here is the RMS of those values over the window. Mean and std are the contents of training_stats.npy.

3 Training statistics: created once, loaded, never recomputed

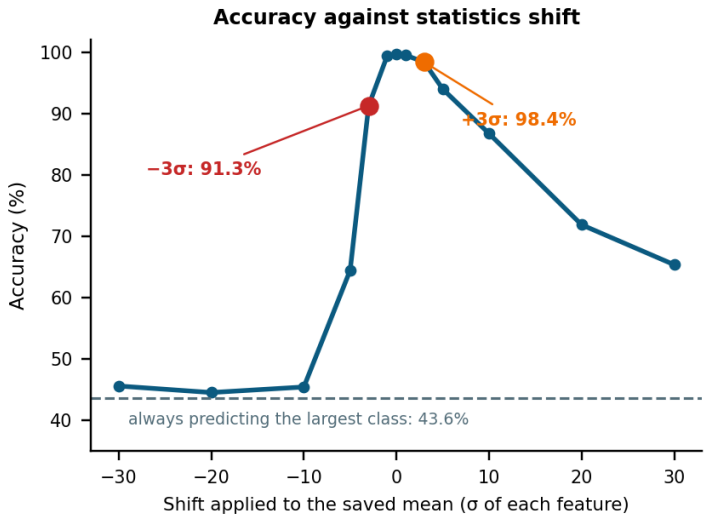
- **Creation (offline):** `python3 preprocessing.py make-stats` generates 10 minutes (600 s) of clean Normal-class data with the Task C1 sensor models, runs it through the pipeline and saves per-feature mean and std (ddof = 1). The 10 minutes yield 57 windows; a 58th would need a sample at t = 600 s.
- **File:** a (2, 6) float64 array; row 0 holds the means, row 1 the standard deviations, in the feature order above.
- **Runtime:** `Normaliser.load()` reads the file. The runtime classes have no fit or update method, the arrays are read-only, and tests check that the runtime source never calls the statistics function or saves a file.
- **Caveat:** windows overlap (30 s windows every 10 s), so the 57 windows are correlated and the standard deviations are uncertain. A longer baseline from real trucks is advisable.

4 Mandatory experiment: inference with correct and shifted statistics

Setup. A small reference classifier (6 → 16 → 3, 163 parameters) stands in for the project model. It was trained five times with different seeds on 4,320 windows from 160 simulated runs and tested on 3,240 windows from 120 separate runs (300 s each, anomaly onset between 20 and 150 s, all four modes). Test classes: Normal 1,413, Warning 744, Critical 1,083. Training and testing both use the saved statistics. "Shifted by 3σ" moves each saved mean by three of its own standard deviations, so every normalised feature moves by 3 (+3σ subtracts 3, −3σ adds 3).

Statistics used	Accuracy (mean ± sd, 5 models)	Change	Normal recall	Warning recall	Critical recall
Correct (saved file)	99.67% ± 0.07	—	99.9%	98.8%	100.0%
Mean shifted +3σ	98.38% ± 0.76	−1.29 points	99.9%	93.7%	99.6%
Mean shifted −3σ	91.28% ± 5.81	−8.38 points	81.5%	98.9%	98.9%
Recomputed from live data (never do this)	43.60%	−56.06 points	—	—	—

Always predicting the largest class would score 43.6%. "Recomputed from live data" normalises each test run with its own windows' mean and std.





Feature shifted by +3σ (others correct)	Physical size of 3σ	Accuracy	Change
temp_std	0.079 °C	98.22%	−1.44 points
temp_mean	0.165 °C	99.29%	−0.38 points
temp_rate_c_per_min	1.09 °C/min	99.32%	−0.35 points
vib_kurtosis	1.59	99.43%	−0.24 points
vib_peak	0.035 g	99.48%	−0.19 points
vib_rms	0.032 g	99.51%	−0.15 points

Findings

- **A 3σ shift costs 1.3 points one way and 8.4 points the other.** The sweep shows accuracy falling steadily with larger shifts, to 86.74% at +10σ and 45.46% at −10σ (no better than always predicting the largest class).
- **The two directions fail differently.** With −3σ, Normal windows look 3σ too high, so 1,308 of 7,065 Normal test windows (18.5%, counted over the five models) were flagged as Warning: false alarms. With +3σ, every window looks 3σ too low, so 169 of 3,720 Warning windows (4.5%) were read as Normal: missed early warnings, the more dangerous error for a cold chain.
- **A small physical shift is enough.** Because the Normal class is tight, 3σ is only 0.17 °C on mean temperature or 0.03 g on vibration RMS. A sensor offset or calibration drift of that size, if not reflected in the saved statistics, moves every input by the same 3σ.
- **Recomputing statistics from live data destroys detection.** Each run is normalised against itself, so the anomaly becomes the new “normal” and accuracy falls to 43.6%. This is why the pipeline only loads the saved file.
- **Spread across models is large at −3σ** (± 5.8 points), so the sensitivity depends on how a particular model places its boundaries. The asymmetry is probably because anomalies only push features upward, so boundaries sit closer to the Normal cluster on the high side. This is an interpretation, not something I tested.
- **Detection stays well inside 90 s.** From fault onset to the first Warning or Critical window, the reference model took on average 18.2 s for temperature drift (max 26 s), 6.6 s for vibration (max 11 s) and 8.7 s for combined (max 12 s), all well inside 90 s, with no run missed.

5 Verification

Check	What it confirms	Result
Moving average	Warm-up and steady-state values; matches numpy convolution	Pass
Features	Known ramp gives 1.2 °C/min; 0.08 °C/s gives 4.8 °C/min; RMS, peak and kurtosis reference values	Pass
Windowing	Ends at 30, 40, 50 s…; step 10 s; 57 windows per 10 min; 30 temperature and 15 vibration samples each	Pass
Filter before features	Window samples equal the moving-averaged stream	Pass
Gaps and bad input	Incomplete windows skipped; duplicate and late messages dropped; unknown stream rejected	Pass
Normaliser	Round trip through .npy; bad shape or zero std rejected; arrays read-only; shift moves z by exactly 3	Pass
Never recomputed	No fit or update method; runtime source never computes or saves statistics; statistics unchanged after live data	Pass
MQTT runtime (paho stub)	Subscribes to temp and vib, publishes features, ignores malformed messages	Pass
Live Mosquitto	Not run: paho-mqtt and Mosquitto are not available in my environment	Please run

6 Assumptions and items to confirm

- **Class labels.** Class 1 is temperature 1–3 °C off setpoint, or a vibration fault (early refrigeration anomaly). Class 2 is more than 3 °C off, or a vibration fault with temperature more than 1 °C off (my reading of "refrigeration unit failure signature"). Please confirm.
- **Meaning of "shifted by 3σ".** I moved the saved mean by 3σ in every feature and report both signs. If you meant shifting the standard deviations too, tell me and I will rerun.
- **Reference classifier.** It is a placeholder. To use your trained model, replace it in `experiments_c2_c3.py` and rerun to refresh these numbers.
- **Data are simulated** with the Task C1 models, so absolute accuracies are optimistic. The statistics should be rebuilt from real Normal-class truck data when available.
- **Definitions.** Standard deviation is the population value; kurtosis is excess kurtosis; the moving average warms up over the first four samples.
- **Door events** are not used, since none of the six specified features comes from them.